

A New Heuristic Method for Simple Plant Location Problem

曾伶玉^{1,2} 吳志盛¹

¹國立中興大學資訊科學與工程學系

²國立中興大學資訊網路與多媒體研究所

lytseng@cs.nchu.edu.tw, phd9209@pop.cs.nchu.edu.tw

摘要

簡易工廠選址問題(Simple Plant Location Problem)是一個典型的 0-1 編碼組合最佳化問題，過去許多學者便針對此問題與其變形發展許多解題方法。本論文提出了一個有效計算成本變動的方法並加入了節省計算時間的概念提出了一個高效的啟發式演算法(heuristic algorithm)。在實驗中証明了此方法的效能，尤其在加入隨機搜尋因子後，其搜尋效果更可比目前許多有效的超啟發式方法(meta-heuristic algorithm)更具有優勢。

一. 簡介

簡易工廠選址問題(英文簡稱 SPLP)在過去許多年便被許多研究者重視與研究，此問題的重要性在於一個大範圍之商業或公共設施的成功或失敗取決於他們的地點，一個良好的設施地點(例如倉庫)可有效的達成供應鏈管理，降低成本或改善客戶滿意度，而此問題在過去的研究中有許多不同的名稱，諸如無容量限制倉庫選址問題(uncapacitated warehouse location problem)或是無容量限制設施選址問題(uncapacitated facility location problem)。

SPLP 問題模型中存在顧客集合 U 與候選工廠集合 F ，主要目標為尋得一個工廠子集合 $s \subset F$ 使得總成本(總運輸成本與工廠建置成本)為最小。由於 SPLP 為一個典型的 NP-complete 問題，因此在過去有許多學者發展了許多不同的方法來求解[17]，而問題特性使得大部分的方法都使用二進制編碼方式，因此早期所發展的啟發式方法不外乎 ADD、DROP、SWAP 三種操作，Kuehn and Hamburger[13]提出了一個二階段的啟發式方法，第一個階段是 ADD 貪婪法，也就是在一開始把所有的工廠全部關閉並且每次都挑選一個能節省最大成本的工廠開啟，直到沒有工廠在被開啟後能節省成本為止，再接再厲執行第二個階段 SWAP 區域搜尋，也就是把任一個關閉與任一個開啟的工廠進行交換(若交換後能節省成本)。此外 Cornuejols G 與 Fisher ML [7]提出另一種啟發式方法為將所有的工廠全部打開並且每次都挑選一個能節省最大成本的工廠關閉，直到沒有工廠在被關閉後能節省成本為止。而其他有效的啟發式方法還有 Lagrangian method[4]與 dual approach[1]

[8]都有被應用到 SPLP，且都有不錯的表現。

啟發式方法通常與問題性質有相當緊密的關聯性，可在很快的時間內尋得可行解，但不保證為最佳解，也因此近年來有許多超啟發式方法也被應用到此問題上，諸如基因演算法[16]、禁制搜尋法[2]、鄰近區域搜尋[9]、path-relinking[19]等，而這些方法中又以禁制搜尋法的表現最為最佳，Michel and Van Hentenryck[18]於 2004 年提出一個簡單的禁制搜尋方法，使用禁制串列來記錄不可被反轉狀態的工廠，並在所有可反轉狀態的工廠中尋找 gains 最大的工廠進行反轉，最後將此反轉的工廠記錄於禁制串列中，若所有工廠都為不可被反轉狀態時，便隨機挑選一個已開啟的工廠並將其關閉。此外 Minghe Sum[20]於 2006 亦使用禁制搜尋法來求解 SPLP，在其論文中並提出 net cost changes 的成本計算方法以加快計算每次反轉時的成本變動。

而上述的這些超啟發方法通常較注重於區域的搜尋效果，因此通常會搭配一些有效的區域搜尋方法進行深度搜尋，因此如何發展一個有效的區域搜尋方法在最佳化的問題領域中是一個非常重要的議題。本論文提出了一個針對 SPLP 非常有效的啟發式區域搜尋方法 EHS(Efficient Heuristic Search)，此方法參考了 Minghe Sum[20]的淨成本變動(net cost changes)的概念，並混用了 ADD、DROP、SWAP 三種操作，實驗數據也證明此方法在測試題目中可將一個隨機產生的初始解改進至區域最佳解。而在本論文也實驗了大量的基準測試問題並與其他的方法的搜尋結果進行比較，証實了此方法的有效性。

二. SPLP 問題定義

在 SPLP 定義中存在顧客集合 $U=\{1, \dots, m\}$ 與候選工廠集合 $F=\{1, \dots, n\}$ ，每個顧客 $i \in U$ 與任一工廠 $j \in F$ 之間存在一條運輸的距離成本 C_{ij} ，此外每個工廠都有一個固定的開啟成本 f_j 。因此一個完整的 SPLP 可定義如下[6]:

$$\text{Min} \quad \sum_{i=1}^m \sum_{j=1}^n C_{ij} x_{ij} + \sum_{j=1}^n f_j p_j \quad (1)$$

Subject to

$$\sum_{j=1}^n x_{ij} = 1 \quad \text{for } i=1, \dots, m \quad (2)$$

$$x_{ij} \leq p_j \quad \text{for } i=1, \dots, m \text{ and } j=1, \dots, n \quad (3)$$

$$x_{ij} \in \{0,1\} \quad \text{for } i=1, \dots, m \text{ and } j=1, \dots, n \quad (4)$$

$$p_j \in \{0,1\} \quad \text{for } j=1, \dots, n. \quad (5)$$

在公式 1 中定義了此問題的目標函式，希望能從工廠集合中挑選部分集合 $s \subset F$ 開啟並根據 s 來計算總花費成本(顧客服務成本與工廠開啟成本)，而基本上我們採用二進制陣列編碼此問題，因此一組完整的編碼可表示成 $s = \{p_1, p_2, \dots, p_n\}$ ，而 p_j 表示每個工廠之狀態，若 $p_i=1$ 則表示第 j 個工廠被開啟，反之則表示該工廠被關閉。此外 x_{ij} 決定顧客 i 該由那個工廠所服務，若 $x_{ij}=1$ 則表示顧客 i 必須由工廠 j 服務，在問題定義中的限制式(2)表示每個顧客最多只能由一個工廠服務，且每個顧客都必須分配到一個工廠。限制式(3)表示每個顧客只能配置給狀態為開啟的工廠。

三. 有效的啟發式搜尋方法

如上所述，一般啟發式方法的操作有 add、drop 與 swap 三種，在過去這三種方法都已有被提出，作法都是逐步移動(單一移步操作)。過去所被發展的 DROP 啟發法只是很單純的將所有工廠都開啟再逐次將可節省成本的工廠關閉，直到沒有任何工廠關閉後可節省成本時停止，但由於 SPLP 是一個 NP-complete 問題，因此若單純的關閉現階段節省最多成本的工廠，可能造成往後走到一個錯誤的方向，因此必須要有 add 操作才能從錯誤的方向導正至正確的路徑。而另一個 ADD 啟發法則與 DROP 啟發法相反，雖然它在第二階段加入了 swap 操作，但 swap 只能夠在現有的工廠數量限制之下尋找較好的選址方案，並無法加入或刪除工廠。而在本篇論文中，我們提出一個新的啟發式方法 Efficient Heuristic Search (EHS)，此方法中有效的結合了 add、drop 與 swap 三種操作，並在每一次的移步中挑選適合的操作，因此較前面所描述的啟發式方法更有彈性。在 EHS 中，每組解的編碼方式為 $s = \{p_1, p_2, \dots, p_3\}$ ， p_j 表示每個工廠之狀態，若 $p_i=1$ 則表示第 j 個工廠被開啟，反之則表示該工廠被關閉，並計算每組解之總成本。

3.1 節省時間之變動成本計算法

在過去的啟發式方法中，都必須以節省最大成本為目標來進行 add、drop 或 swap 操作，而這樣的方法首先面對的問題便是如何有效的計算成本的改變，並找出節省最大成本之工廠來開啟、關閉或交換。

Minghe Sum[20]於論文中提出了 net cost change 的計算方法，在其論文中使用一維矩陣記錄每個工廠之狀態被改變時的成本變動值，而主

要的移步操作(move)以 add 與 drop 二個為主，並提出六個公式用以計算維護矩陣內的成本變動值

在本論文中參考了該方法初始計算 add 變動成本與 drop 變動成本的二個公式，由於我們主要是要計算每個工廠 j 的狀態被改變之後能節省多少成本，因此我們希望矩陣內第 j 個元素若為正值表示第 j 個工廠若被改變狀態則將減少整組解的成本。

首先我們先對一組解 s 定義了二個子集合， s_0 代表所有被關閉的工廠，而 s_1 則為所有被開啟的工廠。此外再為每個顧客 i 定義在 s_1 之中最靠近的工廠 d_i^1 與次近的工廠 d_i^2 。而 X_j 被定義當工廠 j 之狀態由關閉變為開啟時必須供給的顧客集合， Y_j 則被定義當工廠 j 之狀態由開啟變為關閉時原本供給的顧客集合。

$$\Delta^+ O_j = \sum_{i \in X_j} (c_{id_i^1} - c_{ij}) - f_j. \quad (6)$$

$$\Delta^- O_j = f_j - \sum_{i \in Y_j} (c_{id_i^2} - c_{ij}). \quad (7)$$

式子(6)為工廠 j 開啟時所節省的成本，主要計算將 X_j 內的顧客配置給工廠 j 所節省的成本再減去開啟成本，此為 add 操作時需要用到的計算公式。而式子(7)則為工廠 j 關閉時所節省的成本，主要計算工廠 j 關閉所節省的開店成本減去 Y_j 配置給其他工廠所增加的成本，此為 drop 操作時需要用到的計算公式。為了方便比較二種操作的節省成本，計算結果若為正值，則表示該操作可節省成本，反之則為增加成本。此外我們又另外定義 swap 操作時需要用到的式子如下：

$$\bar{Z}_{jk} = \Delta^+ O_j + f_k \quad (8)$$

$$Q_{ij} = \min \left\{ \max \{ c_{id_i^1} - c_{ij}, c_{id_i^1} - c_{id_i^2} \}, 0 \right\}, i \in Y_k \quad (9)$$

$$Z_{jk} = \bar{Z}_{jk} + \sum_{i \in Y_k} Q_{ij}. \quad (10)$$

Minghe Sum[20]於論文中並未提到 swap 的成本計算，而在我們的方法中需要使用到 swap 操作，因此我們定義了式子(8), (9), (10)為 swap 操作的節省成本計算公式。式子(8)為工廠 j 開啟且將工廠 k 關閉時之部分成本計算，此部分只考量 j 開啟時節省的成本與 k 被關閉後節省的開店成本，若 $\bar{Z}_{jk}$ 為正值表示工廠 j 與工廠 $\hat{j}$ 具有 swap 的潛力，將成為一組候選的 swap 操作。而式子(9)為計算當 k 被關閉時，原本屬於 k 之顧客重新配置後所影響的成本。最後的式子(10)為 swap(j, k)之完整成本計算，若 swap(j, k)為候選 swap 操作時，式子(9)與式子(10)才會被計算。相同的若 Z_{jk} 為正值表示此 swap 操作可節省成本，反之則為增加成本。

3.2 EHS 程序

```

Efficient_Heuristic_Search( s )
/*Initialization phase*/
1  Count ← 0;
2  Calculate the saving cost of dropping plant  $k$ ,  $\Delta^-O_k$ , for each plant  $k \in s_1$ 
3  Calculate the saving cost of adding plant  $j$ ,  $\Delta^+O_j$ , for each plant  $j \in s_0$ 
4  Calculate the saving cost of swapping plant  $j$  and plant  $k$ ,  $Z_{jk}$ , and find the
   best swap target  $E_j$  for each plant  $j \in s_0$ 
/*Search phase*/
5  repeat
6    Operator ←  $\emptyset$ ;
7    Update the saving cost of dropping plant  $k$ ,  $\Delta^-O_k$ .
8     $D \leftarrow \arg \max \{ \Delta^-O_j \mid j \in s_1 \}$ ; /*Get the best droppint plant  $D$ */
9    if  $\Delta^-O_D > 0$  then Operator ← Drop(  $D$  );
10   else
11     Update the saving cost of adding a plant,  $\Delta^+O_j$ .
12      $A \leftarrow \arg \max \{ \Delta^+O_j \mid j \in s_0 \}$ ; /*Get the best adding plant  $A$  */
13     Update the saving cost of swapping plant  $j$  and plant  $k$ ,  $Z_{jk}$  and  $E_j$ .
14      $\hat{A} \leftarrow \arg \max_j \{ Z_{j\hat{E}_j} \mid j \in s_0 \}$ ; /*Get the best swapping plant  $\hat{A}$  */
15      $\hat{D} \leftarrow E_{\hat{A}}$ ; /*Get the best swapping plant  $\hat{D}$  */
16     if  $\Delta^+O_{\hat{A}} > Z_{\hat{A}\hat{D}}$  then
17       if  $\Delta^+O_{\hat{A}} > 0$  then Operator ← Add(  $A$  );
18       else if  $Z_{\hat{A}\hat{D}} > 0$  then Operator ← Swap(  $\hat{A}$ ,  $\hat{D}$  );
19   if Operator  $\neq$  TabuOperator then
20     Count ← Count + 1;
21     Execute Operator;
22     TabuOperator ← inverse of Operator;
23   else Operator ←  $\emptyset$ ;
24   if Count >  $n$  then Operator ←  $\emptyset$ ;
25 until Operator =  $\emptyset$ 

```

圖 1. EHS 簡易程式碼

在描述完整的程序之前，我們首先定義 Operator 為一自訂型態，主要用來儲存每次執行的操作項目。操作項目主要有 Drop、Add 與 Swap 三種。每個操作項目都有相對應的工廠被選擇，例如 Drop(D)表示工廠 D 之狀態將從開啟被轉變成關閉，Add(A)表示工廠 A 之狀態將從關閉被轉變成開啟，而 Swap(A, D)則表示將開啟工廠 A 並關閉工廠 D 。

圖 1 為 EHS 的簡易程式碼，在一開始，系統會產生一組解 s ，而在對 s 執行 EHS 之前必須先計算所有需要的移步資訊。首先對所有狀態為開啟之工廠使用式子(7)計算關閉此工廠所節省之成本 (Line 2)，再對所有狀態為關閉的工廠使用式子(6)計算開啟節省成本(Line 3)。最後對每個目前狀態為關閉的工廠 j 使用式子(10)計算交換所節省之成本並找到最適合的交換對象 E_j (Line 4)。

接下來便進入搜尋“移步操作”的程序，整個程序的步驟為：(一)從 Δ^-O_k 中挑選最大節省成本的擬關閉之工廠 D (Line 8)，若 $\Delta^-O_D > 0$ 表示將工廠 D 關閉可改善 s ，因此 EHS 將設定“移步操作”為 Drop(D) (Line 9)。(二)若 $\Delta^-O_D < 0$ ，則 EHS 必須從 Δ^+O_j 中挑選最大節省成本的擬開啟之工廠 A (Line 11)，而在挑選最佳開啟工廠 A 之前必須先更新 Δ^+O_j (Line 11)。除此之外，EHS 將從所有候選 swap 操作中選取最佳 swap 組合之工廠 A 與

$\hat{D}$ (Line 14~15)，相同的在尋找 swap($\hat{A}, \hat{D}$)之前必須先更新每組候選 swap 操作的節省成本(Line 13)，若 swap($\hat{A}, \hat{D}$)所能節省的成本比 Add(A)更多，則“移步操作”將被設定為 swap($\hat{A}, \hat{D}$)，反之則為 Add(A)。值得注意的是 EHS 只會執行改善成本的操作，因此不管最後的操作項目是 Drop、Add 或 Swap，其節省的成本必定大於 0 才會被執行 (Line 16~18)，反之則表示 EHS 已無法再改善 s ，搜尋程序將會被結束。

在 EHS 執行操作之前，會先判斷目前的操作是否會回復前一次的移步。也就是 EHS 將禁止每一次的移步在下次的操作被回復 (Line 19)，為了達到上述的目的，EHS 在每次執行移步將使用 TabuOperator 記錄此“移步操作”的反向狀態，例如本次的 Operator 為 Add(A)，則 TabuOperator 將記錄 Drop(A) (Line 22)，此作法的主要目的是為了禁止“短周期”的重覆移步，所謂的短周期指的是上一步的操作在下一次被回復，例如 Drop(i) $\rightarrow$ Add(i) 或是 Swap(i, j) $\rightarrow$ Drop(i) $\rightarrow$ Add(j)。但在某些情況下可能會發生“長周期”的重覆移步現象，例如 Drop(i) $\rightarrow$ Add(j) $\rightarrow$ Add(k) $\rightarrow$ Add(i) $\rightarrow$ Drop(k) $\rightarrow$ Drop(j)。由於無法預期“長周期”的周期長短，因此在方法中，我們限制每組解 s 的“移步操作”次數最多只能執行 n 次 (n 為題目的工廠數量) (Line 24)。

每次執行完”移步操作”後將會改變 s 中部分工廠的狀態與顧客的配置，為了有效的維護節省成本 Δ^+O_j 、 Δ^-O_k 與 Z_{jk} ，EHS 必須記錄每次移步後狀態被改變的工廠與配置有變動的顧客，使得 EHS 在每次的移步後只需更新狀態有改變的工廠之節省成本。

在工廠關閉節省成本 Δ^-O_k 的更新部份，EHS 只會更新顧客配置有變動的工廠(Line 7)，由於在 EHS 的運作迴圈中，每次必定先搜尋 s 中可被 Drop 的工廠，因此 Δ^-O_k 的更新是每次執行完”移步操作”後都必須要被執行的。另一方面較重要的節省成本維護在於工廠開啟節省成本 Δ^+O_j 與工廠交換節省成本 Z_{jk} ，如圖 1.所示 EHS 只有在 $\Delta^-O_D < 0$ 時才會挑選 add 操作或 swap 操作，由於此步驟在 EHS 中並非每次都會被執行，因此節省成本 Δ^+O_j 與 Z_{jk} 的資訊必須在搜尋 add 操作或 swap 操作之前更新 (Line 11 與 13)即可。

在更新 Δ^+O_j 時將有二種狀況會發生，第一種狀況是工廠 j 的開啟節省成本為 0 時(表示工廠 j 原本是開啟狀態，所以我們沒有計算過 Δ^+O_j ，只有計算過 Δ^-O_j)，但工廠 j 在移步操作過程中狀態被轉換為關閉，因此，我們將使用式子(6)計算工廠 j 的開啟成本。反之則為第二種狀況，也就是工廠 j 的節省成本不為 0 時，EHS 只會利用配置有被改變之顧客來更新工廠 j 被開啟後所節省之成本。

最後由於 Z_{jk} 的計算必須參考的二個主要元素為(1)工廠 j 的開啟節省成本與(2)工廠 k 所支援的顧客若重新配置所變動的成本。因此需要被重新計算交換成本的工廠有二類，第一類為開啟節省成本被更新的工廠 j ，第二類為候選交換對象 E_j 所支援的顧客被變動的工廠 j 。而其他工廠只需要重新尋找是否有更適合 swap 的工廠 E_j 。

EHS 有效的混合 Add、Drop 與 Swap 操作，使得整個方法具有極大的彈性，這部份的搜尋能力也是過去 Drop 啟發式搜尋法與 Add 啟發式搜尋法所沒有的。此外我們更發展快速的節省成本計算方法，在每次尋找最佳移步操作時，只需更新有變動的工廠之狀態移轉成本即可，這也使得 drop、add 與 swap 之節省成本計算操作更有效率。

四. 實驗結果

為了測試 EHS 的效率，我們使用 C++ 撰寫程式碼，並使用 Intel Core2 Duo 2.33 GHz 的 CPU 搭配 windows XP 的作業系統平台進行實驗(實驗只使用一顆 cpu 的運算能力)。

4.1 實驗題組

我們的方法實驗了 Uflib 網站[10][11][12]所放置的大部分類型的題組並將這些不同的來源的題目做成一樣的文字檔格式，而這些題組都有不

同的特性，分別說明如下：

- ORLIB: 這些 instances 被發布於 Beasley 的 OR-Library 上面，原始的檔案是由 Beasley[3][4]於 1993 年所發表的論文中提出，此題組分為 4 個 classes 共 15 題。
- M*: 這些 instance 是 Kratica et al.[16] 於 2001 年使用基因演算法求解 SPLP 時所提出。這組題目共有 30 題，分為六個不同大小的子題組(100、200、300、500、1000 與 2000)，而每個子題組有 5 個題目。
- BK: 由 Bilde 與 Krarup[5]於 1977 年的論文中所提出，顧客與工廠之間的距離是使用 uniform 分布隨機產生於 $[0, 1000]$ 之間，此題組共有 220 題，題組 size 最小的為 30×30 ，而最大的為 100×100 。
- GR: 由 Galvao 與 Raggi[10]使用圖形化方式產生的題組，共有 50 題，題組大小分為五個類別(50, 70, 100, 150 與 200)，每個類別有十個小題。
- FPP: 此題組是人工產生的特製題組，由 Kochetov and Ivanenko[14][15]於第 5 屆 MIC 研討會所發表的論文中提出，並將此題組放在其網站上。此題組共有 40 題又分為 FPP11($n=11$ and $m=133$)與 FPP17($n=17$ and $m=307$)二大類。由於題組經過特別的設計，因此在所有題組當中也是屬於較難的題目。
- GAP: 此題組也是由 Kochetov and Ivanenko[14][15]於第 5 屆 MIC 研討會所發表的論文中提出，這些題目共分為三個類別，分別為 GAPA、GAPB 與 GAPC，每個類別有 30 個題目，其題組特性有較大的 duality gaps，因此對 dual-base 的方法而言是較困難的題目。

4.2 EHS 效能實驗

表 1 測試 ORLIB 與 M*二組基準測試題目，前三個欄位為題目的基本資料(名稱、大小、最佳解)。第四個欄位與第五個欄位主要測試 Drop-heuristic 與 EHS 之搜尋效能，每個題目在實驗時使用的初始解是將每個工廠都開啟以測試方法的效能，根據實驗結果所示，Drop-heuristic 共有 17 題無法找到最佳解，我們進一步分析發現這 17 題中有部分題目與最佳解的差異只有 1 個 neighborhood 的差距，這意味者 drop 方法不能進行有效的 local search。而我們所提的 heuristic 在固定初始解的狀況下只有三個題目無法找到最佳解。

進一步分析發現這三個題目最少必須要有二個 swap 操作才能移動到最佳解。如表 2 所示我們分別列出 cap133、capc 與 mo3 的最佳解與 EHS 所尋得的解，而每個題目要由 EHS 所尋得的解移動到最佳解必須要同時執行多次的移步，例如 cap133 必須要同時執行 swap(6,11) 與 swap(25,15)，而 capc 則必須要同時執行三次的 swap。由於我們的方法每次只能執行一次的操作

表 1. Drop 啟發式方法與 EHS 之效能測試

題目	大小 (m x n)	OPT	Drop-Heuristic (all open)	EHS (all open)	EHS (random open)		
			Dev. from OPT	Dev. from OPT	AVG Dev	Reach	AVG Time
Cap71	50x16	932615.75	0.00	0.00	0.00	10	0.000
Cap72		977799.40	0.00	0.00	0.00	10	0.000
Cap73		1010641.4	0.00	0.00	0.00	10	0.000
Cap74		1034976.9	0.26	0.00	0.00	10	0.000
Cap101	50x25	796648.44	0.00	0.00	0.00	10	0.000
Cap102		854704.20	0.09	0.00	0.00	10	0.000
Cap103		893782.11	0.00	0.00	0.00	10	0.000
Cap104		928941.75	0.61	0.00	0.00	10	0.000
Cap131	50x50	793439.56	0.00	0.00	0.00	10	0.000
Cap132		851495.32	0.09	0.00	0.00	10	0.000
Cap133		893076.71	0.08	0.08	0.00	10	0.000
Cap134		928941.75	0.61	0.00	0.00	10	0.000
CapA	1000x100	17156454.	6.95	0.00	0.00	10	0.084
CapB		12979071.	1.04	0.00	0.00	10	0.100
CapC		11505594.	1.24	0.26	0.00	10	0.109
MO1	100x100	1156.91	0.53	0.00	0.00	10	0.09
MO2		1227.67	0.00	0.00	0.00	10	0.08
MO3		1286.37	0.94	0.14	0.00	10	0.09
MO4		1177.88	0.16	0.00	0.00	10	0.09
MO5		1147.60	0.00	0.00	0.00	10	0.08
MP1	200x200	2460.10	0.00	0.00	0.00	10	0.041
MP2		2419.32	0.00	0.00	0.00	10	0.042
MP3		2498.15	0.00	0.00	0.00	10	0.042
MP4		2633.56	0.45	0.00	0.00	10	0.044
MP5		2290.16	0.00	0.00	0.00	10	0.044
MQ1	300x300	3591.27	0.00	0.00	0.00	10	0.109
MQ2		3543.66	0.04	0.00	0.00	10	0.109
MQ3		3476.81	0.85	0.00	0.00	10	0.120
MQ4		3742.47	0.00	0.00	0.00	10	0.111
MQ5		3751.33	0.00	0.00	0.00	10	0.113
MR1	500x500	2349.86	0.00	0.00	0.00	10	0.362
MR2		2344.76	0.00	0.00	0.00	10	0.414
MR3		2183.24	0.00	0.00	0.00	10	0.400
MR4		2433.11	0.00	0.00	0.00	10	0.377
MR5		2344.35	0.00	0.00	0.00	10	0.372
MS1	1000x1000	4378.63	0.00	0.00	0.00	10	2.489
MS2		4658.35	0.05	0.00	0.00	10	2.586
MS3		4659.16	0.67	0.00	0.00	10	2.858
MS4		4536.00	0.00	0.00	0.00	10	2.427
MS5		4888.91	0.00	0.00	0.00	10	2.172
MT1	2000x2000	9176.51	0.00	0.00	0.00	10	14.986
MT2		9618.85	0.00	0.00	0.00	10	13.878
MT3		8781.11	0.00	0.00	0.00	10	14.654
MT4		9225.49	0.00	0.00	0.00	10	14.855
MT5		9540.67	0.00	0.00	0.00	10	13.775

(add、drop 或 swap)，因此若要同時執行多次的 swap 操作是無法達到的，但也由此實驗結果可證明 EHS 可使一組解移動至區域最佳解。

表 2. EHS 求得的解與 OPT 之差異

題目	方法	解品質
Cap133	OPT	<u>6</u> , <u>23</u> , <u>25</u> , <u>27</u> , <u>34</u> , <u>45</u> , <u>46</u> , <u>49</u>
	EHS(All Open)	<u>11</u> , <u>15</u> , <u>23</u> , <u>27</u> , <u>34</u> , <u>45</u> , <u>46</u> , <u>49</u>
Capc	OPT	<u>6</u> , <u>14</u> , <u>24</u> , <u>35</u> , <u>53</u> , <u>70</u> , <u>79</u> , <u>81</u> , <u>89</u>
	EHS(All Open)	<u>6</u> , <u>9</u> , <u>14</u> , <u>24</u> , <u>35</u> , <u>48</u> , <u>66</u> , <u>70</u> , <u>79</u>
MO3	OPT	<u>6</u> , <u>39</u> , <u>48</u>
	EHS(All Open)	<u>7</u> , <u>38</u> , <u>39</u> , <u>100</u>

在表 1 的實驗數據第 5 個欄位中，我們只使用一組相同的解(開啟所有的工廠)執行 EHS 方法，因此最後都將會產生相同的解答，假設我們能在解空間中隨機產生多組不同的初始解，則將有機會從不同的路徑到達最佳解。為了證明我們的想法，我們對每個題目使用隨機的方式產生 10 個初始解，再針對每個初始解執行 EHS，表 1 第 6 個欄位是對每個題目實驗了十次的平均 deviation，第 7 個欄位為 10 次實驗中找到最佳解的次數，第 8 個欄位為平均的實驗時間。實驗證明我們的方法若同時使用 10 個隨機的初始解可在 ORLIB 與 M*二組基準測試題目找出全部的最佳解，且所耗費的時間也非常少。

而 EHS 在表 3 的實驗中，我們使用了 4 種不同的初始解數量(10, 50, 100, 1000)，並依問題的難易程度實驗不同的參數，而每個題目我們都實驗 50 次並記錄尋得的最佳解，最後呈現每個題組的 Best AVG deviation ((尋得最佳目標值-目前已知最佳目標值)/目前已知最佳目標值*100)與平均執行時間。在實驗數據中，ORLIB、M*、BK 與 GR 四組題目在種子數設定為 10 時對所有的問題實驗

50 次便能尋得目前已知最佳解，特別是 ORLIB、M*與 GR 三個題組的每個題目均能 50 次全找到，而在 BK 題組中則有幾個題組是比較困難的，因此在種子數設定為 100 時才能全找到最佳解。

此外在 FPP 與 GAP 二個題組中雖然最大的題目是 307X307，但由於題目經過特別的設計使得困難度較高，由於題目的 duality gaps 比較大，因此若無法尋得最佳解，則 AVG deviation 會非常高。在此二組題目中，我們則實驗了四個參數以了解我們的方法是否透過不同的初始解數量會如何影響求解品質。在 FPP 的問題中若將初始解數量調至 100 時，可尋得大部份題目的最佳解，但 50 次的實驗中尋得所有最佳解的數量只有 5 次。而在 1000 個初始解數量時，EHS 能保證每個題目均能尋得最佳解，並且在 50 次的實驗中尋得所有最佳解的數量為 35，這顯示 EHS 在 Seed 數量足夠的情況下的確能有效的求解困難題目。另一方面在 GAP 的題組中，我們發現此題組的設計較 FPP 困難，雖然題目最大為 100x100，但是當種子數設定為 1000 時 EHS 依然無法在 50 次的實驗中保證能尋得所有題目的最佳解(在本次實驗中有 21 題無法尋得，但根據我們的實驗發現 EHS 有能力尋得所有題目的最佳解，只是無法在同一次的實驗中全部尋得)。

最後在數據比較的部分，由於比較的平台不同，因此我們僅列出實驗數據以供參考，在求解品質的部分 EHS 與 Hybrid[19]在 ORLIB、M*與 GR 三個題組均能以最少的時間求得所有題目的最佳解，而 Simple Tabu[18]則有少部份題目無法尋得。而在 BK 題組中 EHS 則明顯勝出。此外在 FPP 與 GAP 題組的實驗，EHS 則有能力在很少的初始解數量下求得較好的品質，並有能力隨著增加初始解數量而增加求得每個題目的最佳解的機率。

表 3. EHS 與其他方法於六個基準測試問題之實驗數據比較

Class \ Method		ORLIB		M*		GR		BK		FPP		GAP	
		Avg D	Time	Avg D	Time	Avg D	Time	Avg D	Time	Avg D	Time	Avg D	Time
Hybrid	Iteration=8, Elite=5	0.000	0.048	0.004	2.196	0.000	0.090	0.028	0.087	69.370	1.630	9.573	0.348
	Iteration=32, Elite=10	0.000	0.170	0.000	7.860	0.000	0.320	0.002	0.280	33.375	7.660	5.953	1.640
	Iteration=2048, Elite=80	-	-	-	-	-	-	-	-	0.000	330.88	0.820	92.090
Simple	500 Non-improving	0.028	0.160	0.011	1.750	0.100	0.160	0.071	0.155	95.711	0.650	15.901	0.259
Tabu	64000 Non-improving	-	-	-	-	-	-	-	-	71.150	52.080	6.350	22.430
EHS	10 Random Seed	0.000	0.019	0.000	0.919	0.000	0.024	0.032	0.003	46.948	0.042	13.783	0.007
	50 Random Seed	-	-	-	-	-	-	0.001	0.016	3.963	0.212	8.661	0.034
	100 Random Seed	-	-	-	-	-	-	0.000	0.032	0.298	0.425	7.063	0.068
	1000 Random Seed	-	-	-	-	-	-	-	-	0.012	4.462	2.930	0.737
	4000 Random Seed	-	-	-	-	-	-	-	-	0.003	17.532	1.217	3.421
	16000 Random Seed	-	-	-	-	-	-	-	-	0.000	67.358	0.559	20.301

在目前的六個題組中，只有 BK 題組中的部份題目(D9 與 D10 題組)發生”長周期”移步操作循環的情況，而其他的題目(包括後面的二個大題組之所有題目)均無此現象。根據我們的觀察，D9 與 D10 的所發生的長周期之 move 循環長度均為 3，也就是發生的循環情況為 $\text{swap}(i, j) \rightarrow \text{swap}(j, k) \rightarrow \text{swap}(k, i)$ ，雖然這樣的狀況可使用特定案例來處理，但我們不希望 EHS 因其他長度的周期移步操作循環而導致無法正常運作。

五. 結論

本論文中對 SPLP 提出了一個有效的啟發式演算法 EHS，經由實驗證明此方法深度搜尋的能力的確對解空間呈現一個大谷型的問題有非常好的求解效率。而另一方面對較複雜的問題則將需要依賴有效的廣度搜尋(Global Search)，而在本論文中我們在廣大的解空間隨機產生大量的初始解數量以模擬廣度搜尋之效果，也的確在大部分的基準測試問題成功的求解。

EHS 的發展將使得 SPLP 在未來的研究中有一個非常有效區域搜尋方法可使用，而未來的研究方向，我們將再更進一步的探討廣度搜尋方法對 EHS 的影響，畢竟一個有效的最佳化問題之求解需要平衡廣度搜尋與深度搜尋的能力。

六. 誌謝

本研究承蒙國科會計劃(NSC 96-2628-E-0058-074-MY3)之資助，特此致謝

參考文獻

- [1] S. Ahn, C. Cooper, G. Cornuejols and A.M. Frieze, Probabilistic analysis of a relaxation for the k-median problem. *Mathematics of Operations Research* 13, (1998), pp. 1-31.
- [2] K.S. Al-Sultan and M.A. Al-Fawzan MA, A tabu search approach to the uncapacitated facility location problem. *Annals of Operations Research* 86, (1999), pp. 91-103.
- [3] J. Beasley, OR-Library: Distributing test problems by electronic mail. *Journal of the Operational Research Society* 41, (1990), pp. 1069-1072, <http://mscmga.ms.ic.ac.uk/info.html>.
- [4] J.E. Beasley, Lagrangean heuristics for location problems. *European Journal of Operational Research* 65, (1993), pp. 383-399.
- [5] O. Bilde and J. Krarup, Sharp lower bounds and efficient algorithms for the simple plant location problem. *Annals of Discrete Mathematics* 1, (1977), pp. 79-97.
- [6] G. Cornuejols, G.L. Nemhauser and L.A. Wolsey, The uncapacitated facility location problem. in: P.B. Mirchandani, R.L. Francis (Eds.) *Discrete Location Theory*, Wiley-Interscience, New York, (1990), pp. 119-171.

- [7] G. Cornuejols, M.L. Fisher and L.A. Wolsey, Location of bank account to optimize float: an analytic study of exact and approximate algorithms. *Management Science* 23, (1977), pp. 789-810.
- [8] D. Erlenkotter, A dual-based procedure for uncapacitated facility location. *Operations Research* 26, (1978), pp. 992-1009.
- [9] D. Ghosh, Neighborhood search heuristics for the uncapacitated facility location problem. *European Journal of Operational Research* 150, (2003), pp. 150-162.
- [10] R.D. Galvao and L.A. Raggi, A method for solving to optimality uncapacitated facility location problems. *Annals of Operations Research* 18, (1989), pp.225-244
- [11] M. Hoefer, UflLib, 2002. <http://www.mpi-sb.mpg.de/units/agl/projects/benchmarks/UflLib>.
- [12] M. Hoefer, Experimental comparison of heuristic and approximation algorithms for uncapacitated facility location. in: *Proceedings of the Second International Workshop on Experimental and Efficient Algorithms (WEA)*, volume 2647 of *Lecture Notes in Computer Science*, Springer-Verlag, (2003), pp. 165-178.
- [13] A.A. Kuehn and M.J. Hamburger, A heuristic program for locating warehouses. *Management Science* 9, 4, (1963), pp. 643-666.
- [14] Y. Kochetov, Benchmarks library, (2003). <http://www.math.nsc.ru/LBRT/-k5/Kochetov/bench.html>.
- [15] Y. Kochetov and D. Ivanenko. Computationally difficult instances for the uncapacitated facility location problem. in: *Proceedings of the 5th Metaheuristics International Conference (MIC)*, (2003), pp. 41:1-41:6
- [16] J. Kratica, D. Tosic, V. Fillipovic and I. Ljubic, Solving the simple plant location problem by genetic algorithm. *RAIRO Operations Research* 35, (2001), pp.127-142.
- [17] J. Krarup and P.M. Pruzan, The simple plant location problem: survey and synthesis. *European Journal of Operational Research* 1, (1983), pp. 81-106.
- [18] L. Michel and P. Van Hentenryck, A simple tabu search for warehouse location. *European Journal of Operational Research* 157, (2004), pp. 576-591.
- [19] M.G.C. Resende and R.F. Werneck, A hybrid multistart heuristic for the uncapacitated facility location problem. *European Journal of Operational Research* 174, (2006), pp. 54-68.
- [20] M. Sun, Solving the uncapacitated facility location problem using tabu search. *Computers & Operations Research* 33, (2006), pp. 2563-2589.